COMPUTATIONS FOR *MAZUR'S KNOT AND THE OCTAHEDRON*

JACK CALCUT AND YANGYANG DU

Abstract. We provide the code, complete output, and a brief analysis for every computer computation used in Sections 6–8 of our paper *Mazur's Knot and the Octahedron*.

1. Introduction

This is a companion paper to *Mazur's Knot and the Octahedron* [CD26]. It contains the code, complete output, and a brief analysis for every computer calculation used in Sections 6–8 of [CD26]. Together with the GitHub repository, this document makes those computations fully reproducible. All computations were performed using SageMath 10.7, SnapPy 3.3.2, and GAP 4.14.0. The code is available at
https://github.com/yyadu/Mazur-manifold-computation.

We use the following notation consistent with our paper [CD26]:

K	Mazur knot in $S^1 \times S^2$
X	Mazur knot exterior
μ_K	Oriented meridian of K in ∂X
λ_K	Oriented longitude of K in ∂X
$X(p,q)$	Dehn filling of X using slope $(p,q) \leftrightarrow p/q$
M_k	Mazur 4-manifold with framing $k \in \mathbb{Z}$
∂M_k	Boundary 3-manifold of M_k
J	Jester knot in $S^1 \times S^2$
Y	Jester knot exterior
μ_J	Oriented meridian of J in ∂Y
λ_J	Oriented longitude of J in ∂Y
$Y(p,q)$	Dehn filling of Y using slope $(p,q) \leftrightarrow p/q$
N_k	Jester 4-manifold with framing $k \in \mathbb{Z}$
∂N_k	Boundary 3-manifold of N_k

Acknowledgment

The authors are grateful to Nathan Dunfield for invaluable conversations on SnapPy.

2. Mazur and Jester Knot Exteriors

The Mazur knot $K \subset S^1 \times S^2$ and the Jester knot $J \subset S^1 \times S^2$ are shown in Figure 1. Equipping K and J with the framing $k \in \mathbb{Z}$ yields the surgery diagrams for $\partial M_k = X(k,1)$ and $\partial N_k = Y(k,1)$ respectively in Figure 1.

Date: July 31, 2026.

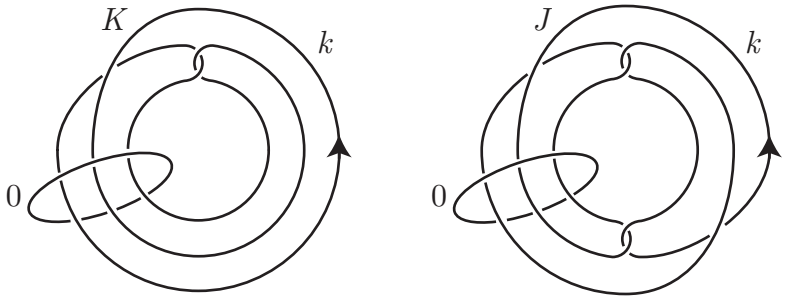

FIGURE 1. Surgery diagrams for $\partial M_k = X(k,1)$ (left) and $\partial N_k = Y(k,1)$ (right).

We first import the link diagrams into SnapPy using the PLink Editor.

```
import snappy
# Opens the PLink Editor to draw links
M = snappy.Manifold()
```

```
Starting the link editor.
Select Tools->Send to SnapPy to load the link complement.
Your new Plink window needs an event loop to become visible.
Type "%gui tk" below (without the quotes) to start one.
```

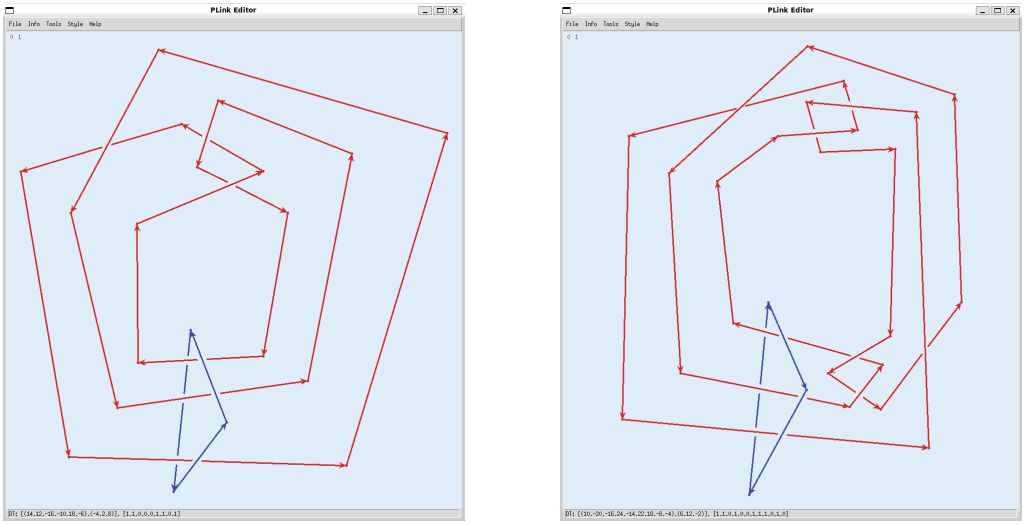

FIGURE 2. PLink Editor link diagrams for Mazur and Jester boundary 3-manifolds, together with their DT codes at the bottom.

The Dowker–Thistlethwaite (DT) codes for the two links in Figure 2 are

- `DT: [(14,12,-16,-10,18,-6),(-4,2,8)], [1,1,0,0,0,1,1,0,1]`
- `DT: [(10,-20,-16,24,-14,22,18,-8,-4),(6,12,-2)], [1,1,0,1,0,0,1,1,1,0,1,0]`

The unknotted blue component is Dehn filled with slope $0/1 \leftrightarrow (0,1)$ to obtain the exterior X of K and Y of J respectively in $S^1 \times S^2$. The cusp shapes of X and Y were computed geometrically in Section 6 of [CD26]. The SnapPy computations below provide independent verification.

In Section 7 ahead, we compute the cusp shapes of the Whitehead link exterior arising both from a standard link in S^3 and a link in $S^1 \times S^2$. The latter computation involves a three-component link. For links with three or more components, standard DT conventions used by the PLink Editor sometimes reorder link components as we discuss further in Section 7.

We compute basic information about X.

```
import snappy
# Mazur knot exterior X
X = snappy.Manifold('DT: [(14,12,-16,-10,18,-6),(-4,2,8)], [1,1,0,0,0,1,1,0,1]')
X.dehn_fill((0,1),1)
X = X.filled_triangulation()
Xid = X.identify()
IsomX = X.symmetry_group()
print("Mazur knot exterior X")
print("Volume =", X.volume())
print("Isometric to census manifold:", Xid)
print("Cusp shape =", X.cusp_info('shape'))
print("")
print("Is X achiral?", IsomX.is_amphicheiral())
print("Order of isometry group =", IsomX.order())
print("Isometry group =", IsomX)
print("Self-isometries are:", IsomX.isometries(), "\n")
print("Isometries X -> m137 are:", X.is_isometric_to(snappy.Manifold('m137'),
    return_isometries=True))
```

```
Mazur knot exterior X
Volume = 3.66386237670888
Isometric to census manifold: [m137(0,0)]
Cusp shape = [3.25000000000000 + 2.25000000000000*I]

Is X achiral? False
Order of isometry group = 2
Isometry group = Z/2
Self-isometries are: [0 -> 0
[1 0]
[0 1]
Extends to link, 0 -> 0
[-1  0]
```

```
[ 0 -1]
Extends to link]

Isometries X -> m137 are: [0 -> 0
[-1 -3]
[ 0 -1]
Extends to link, 0 -> 0
[1 3]
[0 1]
Extends to link]
```

Consequently:

- X is hyperbolic with $\mathrm{vol}(X) \approx 3.66386$
- X is isometric to the 1-cusped census manifold $m137$
- The cusp shape of X is $\tau_X \approx \frac{13}{4} + \frac{9}{4}i$
- X admits no orientation reversing self-isometry
- The isometry group of X has order 2

SnapPy indicates the action of an isometry on cusps using 2×2 integer matrices. Each domain and target cusp torus must be equipped with an ordered homology basis $[\mu_D, \lambda_D]$ and $[\mu_T, \lambda_T]$ respectively. The matrix $A = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$ implies that

$$x\mu_D + y\lambda_D \mapsto (ax + cy)\mu_T + (bx + dy)\lambda_T$$

In other words, the first column of A is the image of μ_D and the second column is the image of λ_D. If a manifold is defined in SnapPy as a link complement, then its cusps are equipped with standard topological meridian–longitude bases. This applies to X and Y, so ∂X is given the basis $[\mu_K, \lambda_K]$ and ∂Y is given the basis $[\mu_J, \lambda_J]$. Otherwise, SnapPy has its own heuristics for choosing bases of cusp tori. This applies to $m137$ and $t12072$.

From these conventions and the above computation we conclude:

- Every self-isometry of X acts trivially on slopes in ∂X
- Every isometry $X \to m137$ acts on slopes by $(p, q) \mapsto (p + 3q, q)$
- (Basis change) $X(k, 1)$ is homeomorphic to $m137(k + 3, 1)$ for each $k \in \mathbb{Z}$

We compute basic information about Y.

```
import snappy
# Jester knot exterior Y
Y = snappy.Manifold('DT: [(10,-20,-16,24,-14,22,18,-8,-4),(6,12,-2)],'
    '[1,1,0,1,0,0,1,1,1,0,1,0]')
Y.dehn_fill((0,1),1)
Y = Y.filled_triangulation()
Yid = Y.identify()
IsomY = Y.symmetry_group()
print("Jester knot exterior Y")
print("Volume =", Y.volume())
```

```
print("Isometric to census manifold:", Yid)
print("Cusp shape =", Y.cusp_info('shape'))
print("")
print("Is Y achiral?", IsomY.is_amphicheiral())
print("Order of isometry group =", IsomY.order())
print("Isometry group =", IsomY)
print("Self-isometries are:", IsomY.isometries(), "\n")
print("Isometries Y -> t12072 are:", Y.is_isometric_to(snappy.Manifold('t12072'),
    return_isometries=True))
```

```
Jester knot exterior Y
Volume = 7.32772475341775
Isometric to census manifold: [t12072(0,0)]
Cusp shape = [6.50000000000000 + 4.50000000000001*I]

Is Y achiral? False
Order of isometry group = 4
Isometry group = Z/2 + Z/2
Self-isometries are: [0 -> 0
[1 0]
[0 1]
Extends to link, 0 -> 0
[1 0]
[0 1]
Extends to link, 0 -> 0
[-1  0]
[ 0 -1]
Extends to link, 0 -> 0
[-1  0]
[ 0 -1]
Extends to link]

Isometries Y -> t12072 are: [0 -> 0
[1 6]
[0 1]
Extends to link, 0 -> 0
[-1 -6]
[ 0 -1]
Extends to link, 0 -> 0
[1 6]
[0 1]
Extends to link, 0 -> 0
[-1 -6]
```

```
[ 0 -1]
Extends to link]
```

Consequently:

- Y is hyperbolic with $\mathrm{vol}(Y) \approx 7.32772$
- Y is isometric to the 1-cusped census manifold $t12072$
- The cusp shape of Y is $\tau_Y \approx \frac{13}{2} + \frac{9}{2}i$
- Y admits no orientation reversing self-isometry
- The isometry group of Y has order 4
- Every self-isometry of Y acts trivially on slopes in ∂Y
- Every isometry $Y \to t12072$ acts on slopes by $(p, q) \mapsto (p + 6q, q)$
- (Basis change) $Y(k, 1)$ is homeomorphic to $t12072(k + 6, 1)$ for each $k \in \mathbb{Z}$

Dunfield [Dun20] determined the exceptional slopes for SnapPy's census manifolds. Exceptional slopes are precisely those yielding nonhyperbolic fillings. For $m137$, the exceptional slopes are

- $(1, 0) \leftrightarrow 1/0 = \infty$ giving $S^1 \times S^2$ upon filling
- $(s, 1)$ where $s \in \{-2, -1, 0, 1\}$

By the basis change, the exceptional slopes of X are

- $(1, 0) \leftrightarrow 1/0 = \infty$ giving $S^1 \times S^2$ upon filling
- $(k, 1)$ where $k \in \{-5, -4, -3, -2\}$

So, the integral exceptional slopes of X are exactly $(k, 1)$ where $k \in \{-5, -4, -3, -2\}$.

For $t12072$, there is a unique exceptional slope

- $(1, 0) \leftrightarrow 1/0 = \infty$ giving $S^1 \times S^2$ upon filling

By the basis change, the exceptional slope of Y is

- $(1, 0) \leftrightarrow 1/0 = \infty$ giving $S^1 \times S^2$ upon filling

So, Y has no integral exceptional slope.

3. Exceptional Fillings of the Mazur Knot Exterior

We distinguish the four exceptional fillings $X(k, 1)$, where $k \in \{-5, -4, -3, -2\}$, pairwise up to homeomorphism. To do so, we count equivalence classes of epimorphisms from the fundamental groups $\pi_1(X(k, 1))$ to alternating groups A_n. The GAP command `GQuotients`(G, H) returns a transversal for the equivalence relation on epimorphisms $G \twoheadrightarrow H$ defined by $f_1 \sim f_2$ if and only if there exists an automorphism ϕ of H such that $\phi \circ f_1 = f_2$.

```
import snappy
from sage.all import gap
# Distinguish exceptional fillings of X
X = snappy.Manifold('DT: [(14,12,-16,-10,18,-6),(-4,2,8)], [1,1,0,0,0,1,1,0,1]')
X.dehn_fill((0,1), 1)
X = X.filled_triangulation()
for n in range(5, 8):
    H = gap(f"AlternatingGroup({n})")
    print(f"Epimorphisms pi_1(X(k,1)) ->> A{n}")
    for k in range(-5, -1):
```

```
        CX = X.copy()
        CX.dehn_fill((k,1), 0)
        G = gap(CX.fundamental_group())
        Q = gap.GQuotients(G, H)
        print(f"  X({k},1): {len(Q)}")
```

```
Epimorphisms pi_1(X(k,1)) ->> A5
  X(-5,1): 1
  X(-4,1): 1
  X(-3,1): 0
  X(-2,1): 0
Epimorphisms pi_1(X(k,1)) ->> A6
  X(-5,1): 0
  X(-4,1): 2
  X(-3,1): 0
  X(-2,1): 0
Epimorphisms pi_1(X(k,1)) ->> A7
  X(-5,1): 0
  X(-4,1): 5
  X(-3,1): 2
  X(-2,1): 0
```

Table 1 collects the results of this computation. Consequently, the fundamental groups $\pi_1(X(k,1))$ are pairwise nonisomorphic for $k \in \{-5,-4,-3,-2\}$. Therefore, the filled manifolds $X(k,1)$ are pairwise nonhomeomorphic for $k \in \{-5,-4,-3,-2\}$.

	A_5	A_6	A_7
$X(-5,1)$	1	0	0
$X(-4,1)$	1	2	5
$X(-3,1)$	0	0	2
$X(-2,1)$	0	0	0

TABLE 1. Numbers of classes of epimorphisms $\pi_1(X(k,1)) \twoheadrightarrow A_n$.

4. Systolic Geodesics

We find all systolic geodesics in the filled manifolds $X(k,1)$ and $Y(k,1)$ for $k \in \mathbb{Z}$. We use a quantitative version of Thurston's hyperbolic Dehn surgery theorem for sufficiently large $|k|$. We cover the remaining finite list of values of k by direct computation.

The quantitative version of Thurston's hyperbolic Dehn surgery theorem that we use requires both the cusp shape and the systole of X and of Y. We computed the cusp shape above. We compute the systole using $m137$ and $t12072$ since SnapPy seems to perform better with difficult computations using census manifolds. As X and $m137$ are isometric, their systoles are equal, and similarly for Y and $t12072$.

```
import snappy
# Systole of m137 and t12072
for name in ["m137", "t12072"]:
    M = snappy.Manifold(name)
    g = M.length_spectrum_alt(count=1, verified=True, bits_prec=100)[0]
    print(f"sys({name}) =", g.length.real())
```

```
sys(m137) = 0.5097911466811016354624?
sys(t12072) = 1.01958229336220327?
```

Consequently, the verified systoles are

$$\begin{aligned}\operatorname{sys}(X) = \operatorname{sys}(m137) &= 0.50979\ldots\\ \operatorname{sys}(Y) = \operatorname{sys}(t12072) &= 1.01958\ldots\end{aligned}$$

A trailing ? in SnapPy's output indicates that the final digit is uncertain while the preceding digits are verified correct. That yields a rigorous small interval containing the actual value.

Since both systoles exceed 0.145, the hypotheses of Futer–Purcell–Schleimer [FPS25, Thm. 3.19] and [FPS25, Thm. 3.23] reduce to requiring only that the normalized slope length be at least 9.97. Using the cusp shapes

$$\tau_X = \frac{13}{4} + \frac{9}{4}i \quad \text{and} \quad \tau_Y = \frac{13}{2} + \frac{9}{2}i$$

one computes that the normalized length of the filling slope $(k, 1)$ is at least 9.97 whenever $|k| \geq 19$ for X and $|k| \geq 28$ for Y. Therefore, by [FPS25, Thm. 3.23], the core of the filling is the unique systolic geodesic in $X(k, 1)$ for $|k| \geq 19$ and in $Y(k, 1)$ for $|k| \geq 28$.

We check whether the core of the filling is the unique systolic geodesic for the remaining finite list of manifolds $X(k, 1)$ with $|k| \leq 18$. We perform the check using $m137$ and translate the results to X using the basis change. We must check the list $m137(s, 1)$ for $-15 \leq s \leq 21$. We compute the larger list $|s| \leq 25$ for overlap.

For $m137$, the ordinary routine `length_spectrum` occasionally succeeds when the verified routine `length_spectrum_alt` cannot certify the hyperbolic structure. Accordingly, we first attempt `length_spectrum` and fall back to `length_spectrum_alt`. For $t12072$, the verified routine succeeds throughout, so only `length_spectrum_alt` is needed.

```
import snappy
# Compare the filling core with the systole and test uniqueness for m137(s,1)
print(f"{'s':^3}  {'core':^12}  {'systole':^12}"
      f"{'core=systole?':^15}  {'unique?':^8}")
print("-" * 56)
for s in range(-25, 26):
    if s in {-2, -1, 0, 1}:
        print(f"{s:3d}  {'---':>12}  {'---':>12}  {'nonhyperbolic':>15}")
        continue
    M = snappy.Manifold(f"m137({s},1)")
```

```
    core = M.cusp_info(0).core_length.real()
    try:
        LS = M.length_spectrum(1.5, include_words=True)
        systole = LS[0].length.real()
        unique = abs(LS[0].length.real() - LS[1].length.real()) > 1e-10
    except RuntimeError:
        LS = M.length_spectrum_alt(count=4, verified=True, bits_prec=200)
        systole = LS[0].length.real().center()
        ordinary = [g for g in LS if g.core_curve is None]
        unique = ordinary[0].length.real().upper() < \
            ordinary[1].length.real().lower()
    core_is_systole = abs(core - systole) < 1e-10
    print(f"{s:3d}  {core:12.8f}  {systole:12.8f}"
      f"{str(core_is_systole):^15}  {str(unique):^8}")
```

```
 s       core        systole    core=systole?   unique?
-------------------------------------------------------
-25    0.02286701    0.02286701     True           True
-24    0.02481388    0.02481388     True           True
-23    0.02701946    0.02701946     True           True
-22    0.02953154    0.02953154     True           True
-21    0.03240940    0.03240940     True           True
-20    0.03572732    0.03572732     True           True
-19    0.03957934    0.03957934     True           True
-18    0.04408588    0.04408588     True           True
-17    0.04940311    0.04940311     True           True
-16    0.05573632    0.05573632     True           True
-15    0.06335964    0.06335964     True           True
-14    0.07264532    0.07264532     True           True
-13    0.08410857    0.08410857     True           True
-12    0.09847779    0.09847779     True           True
-11    0.11680747    0.11680747     True           True
-10    0.14066482    0.14066482     True           True
 -9    0.17244646    0.17244646     True           True
 -8    0.21592633    0.21592633     True           True
 -7    0.27720467    0.27720467     True           True
 -6    0.36629268    0.36629268     True           True
 -5    0.49966077    0.49966077     True           True
 -4    0.70647343    0.59995705     False          True
 -3    1.06960599    0.49916351     False          True
 -2           ---           ---    nonhyperbolic
 -1           ---           ---    nonhyperbolic
  0           ---           ---    nonhyperbolic
```

```
 1          ---          ---    nonhyperbolic
 2   1.43906664   0.36613070     False         True
 3   0.85704045   0.56397216     False         True
 4   0.59110354   0.59110354      True         True
 5   0.42605455   0.42605455      True         True
 6   0.31743363   0.31743363      True         True
 7   0.24383067   0.24383067      True         True
 8   0.19241222   0.19241222      True         True
 9   0.15537015   0.15537015      True         True
10   0.12791980   0.12791980      True         True
11   0.10706453   0.10706453      True         True
12   0.09087339   0.09087339      True         True
13   0.07806528   0.07806528      True         True
14   0.06776650   0.06776650      True         True
15   0.05936618   0.05936618      True         True
16   0.05242748   0.05242748      True         True
17   0.04663167   0.04663167      True         True
18   0.04174201   0.04174201      True         True
19   0.03757975   0.03757975      True         True
20   0.03400801   0.03400801      True         True
21   0.03092051   0.03092051      True         True
22   0.02823379   0.02823379      True         True
23   0.02588158   0.02588158      True         True
24   0.02381071   0.02381071      True         True
25   0.02197816   0.02197816      True         True
```

Consequently, the following hold for $m137(s, 1)$:

- For $s \notin \{-4, -3, \dots, 3\}$, the core of the filling is the unique systolic geodesic
- For $s \in \{-4, -3, 2, 3\}$, the systolic geodesic is not the core of the filling but is unique
- $s \in \{-2, -1, 0, 1\}$ are the exceptional slopes

Therefore, the following hold for $X(k, 1)$:

- For $k \notin \{-7, -6, \dots, 0\}$, the core of the filling is the unique systolic geodesic
- For $k \in \{-7, -6, -1, 0\}$, the systolic geodesic is not the core of the filling but is unique
- $k \in \{-5, -4, -3, -2\}$ are the exceptional slopes

We check whether the core of the filling is the unique systolic geodesic for the remaining finite list of manifolds $Y(k, 1)$ with $|k| \leq 27$. We perform the check using $t12072$ and translate the results to Y using the basis change. We must check the list $t12072(s, 1)$ for $-21 \leq s \leq 33$. We compute the larger list $|s| \leq 35$ for overlap.

```
import snappy
# Compare the filling core with the systole and test uniqueness for t12072(s,1)
print(f"{'s':^3}  {'core':^12}  {'systole':^12}"
      f"{'core=systole?':^15}  {'unique?':^8}")
```

```
print("-" * 56)
for s in range(-35, 36):
    M = snappy.Manifold(f"t12072({s},1)")
    core = M.cusp_info(0).core_length.real()
    LS = M.length_spectrum_alt(count=4, verified=True, bits_prec=200)
    systole = LS[0].length.real()
    core_is_systole = abs(core - systole.center()) < 1e-10
    unique = systole.upper() < LS[1].length.real().lower()
    print(f"{s:3d}  {core:12.8f}  {systole.center():12.8f}"
        f"{str(core_is_systole):^15}  {str(unique):^8}")
```

```
 s       core         systole    core=systole?   unique?
--------------------------------------------------------
-35    0.02334634    0.02334634     True           True
-34    0.02473530    0.02473530     True           True
-33    0.02625116    0.02625116     True           True
-32    0.02790974    0.02790974     True           True
-31    0.02972941    0.02972941     True           True
-30    0.03173155    0.03173155     True           True
-29    0.03394118    0.03394118     True           True
-28    0.03638771    0.03638771     True           True
-27    0.03910591    0.03910591     True           True
-26    0.04213709    0.04213709     True           True
-25    0.04553065    0.04553065     True           True
-24    0.04934594    0.04934594     True           True
-23    0.05365487    0.05365487     True           True
-22    0.05854504    0.05854504     True           True
-21    0.06412408    0.06412408     True           True
-20    0.07052518    0.07052518     True           True
-19    0.07791465    0.07791465     True           True
-18    0.08650197    0.08650197     True           True
-17    0.09655357    0.09655357     True           True
-16    0.10841174    0.10841174     True           True
-15    0.12252094    0.12252094     True           True
-14    0.13946474    0.13946474     True           True
-13    0.16001802    0.16001802     True           True
-12    0.18522130    0.18522130     True           True
-11    0.21648661    0.21648661     True           True
-10    0.25574812    0.25574812     True           True
 -9    0.30567411    0.30567411     True           True
 -8    0.36996287    0.36996287     True           True
 -7    0.45375527    0.45375527     True           True
 -6    0.56420104    0.56420104     True           True
```

-5	0.71087754	0.71087754	True	True
-4	0.90188600	0.90188600	True	True
-3	1.11171893	0.77858590	False	True
-2	1.24807978	0.67096709	False	True
-1	1.29436872	0.61977976	False	True
0	1.29436872	0.61870851	False	True
1	1.24807978	0.66785203	False	True
2	1.11171893	0.77442060	False	True
3	0.90188600	0.90188600	True	False
4	0.71087754	0.71087754	True	True
5	0.56420104	0.56420104	True	True
6	0.45375527	0.45375527	True	True
7	0.36996287	0.36996287	True	True
8	0.30567411	0.30567411	True	True
9	0.25574812	0.25574812	True	True
10	0.21648661	0.21648661	True	True
11	0.18522130	0.18522130	True	True
12	0.16001802	0.16001802	True	True
13	0.13946474	0.13946474	True	True
14	0.12252094	0.12252094	True	True
15	0.10841174	0.10841174	True	True
16	0.09655357	0.09655357	True	True
17	0.08650197	0.08650197	True	True
18	0.07791465	0.07791465	True	True
19	0.07052518	0.07052518	True	True
20	0.06412408	0.06412408	True	True
21	0.05854504	0.05854504	True	True
22	0.05365487	0.05365487	True	True
23	0.04934594	0.04934594	True	True
24	0.04553065	0.04553065	True	True
25	0.04213709	0.04213709	True	True
26	0.03910591	0.03910591	True	True
27	0.03638771	0.03638771	True	True
28	0.03394118	0.03394118	True	True
29	0.03173155	0.03173155	True	True
30	0.02972941	0.02972941	True	True
31	0.02790974	0.02790974	True	True
32	0.02625116	0.02625116	True	True
33	0.02473530	0.02473530	True	True
34	0.02334634	0.02334634	True	True
35	0.02207058	0.02207058	True	True

Consequently, the following hold for $t12072(s, 1)$:

- For $s \notin \{-3, -2, \ldots, 3\}$, the core of the filling is the unique systolic geodesic

- For $s \in \{-3, -2, \ldots, 2\}$, the systolic geodesic is not the core of the filling but is unique
- For $s = 3$, the core of the filling is a systolic geodesic but it is not unique

Therefore, the following hold for $Y(k, 1)$:

- For $k \notin \{-9, -8, \ldots, -3\}$, the core of the filling is the unique systolic geodesic
- For $k \in \{-9, -8, \ldots, -4\}$, the systolic geodesic is not the core of the filling but is unique
- For $k = -3$, the core of the filling is a systolic geodesic but it is not unique

5. Drilling Systolic Geodesics

Drilling unique systolic geodesics is a powerful method for producing topological invariants. We drill the unique noncore systolic geodesics for the hyperbolic filled manifolds $X(k, 1)$ and $Y(k, 1)$. For $Y(-3, 1)$, we show there are exactly two systolic geodesics and drill each of them. We compute with $m137(s, 1)$ and $t12072(s, 1)$ and translate the results to $X(k, 1)$ and $Y(k, 1)$ using the basis changes.

```
import snappy
# Drill the unique systolic geodesic for the remaining
# hyperbolic fillings of m137(s,1)
print(f"{'s':^3}  {'systole':^12}  {'systole drilled manifold'}")
print("-" * 82)
for s in [-4,-3,2,3]:
    M = snappy.Manifold(f"m137({s},1)")
    D = M.drill(0)
    D.dehn_fill((1,0),1)
    F = D.filled_triangulation([0])
    g = F.length_spectrum_alt(count=1, verified=True, bits_prec=200)[0]
    H = F.drill_word(g.word, verified=True, bits_prec=200)
    print(f"{s:3d}  {g.length.real().center():12.8f}  {H.identify()}")
```

```
 s     systole     systole drilled manifold
----------------------------------------------------------------------------------
 -4    0.59995705  [m081(0,0)]
 -3    0.49916351  [m011(0,0)]
  2    0.36613070  [m004(0,0), 4_1(0,0), K2_1(0,0), K4a1(0,0), otet02_00001(0,0)]
  3    0.56397216  [m038(0,0)]
```

```
import snappy
# Drill the unique systolic geodesic for the remaining
# hyperbolic fillings of t12072(s,1)
print(f"{'s':^3}  {'systole':^12}  {'systole drilled manifold'}")
print("-" * 67)
for s in [-3,-2,-1,0,1,2]:
```

```
    M = snappy.Manifold(f"t12072({s},1)")
    g = M.length_spectrum(2.0, include_words=True)[0]
    H = M.drill_word(g.word)
    print(f"{s:3d}  {g.length.real():12.8f}  {H.identify()}")
```

```
 s     systole     systole drilled manifold
-----------------------------------------------------------------
 -3    0.77858590  [v3519(0,0)]
 -2    0.67096709  [v3200(0,0)]
 -1    0.61977976  [v2919(0,0)]
  0    0.61870851  [v2911(0,0)]
  1    0.66785203  [v3184(0,0)]
  2    0.77442060  [v3505(0,0), 8_21(0,0), K7_127(0,0), K8n2(0,0)]
```

Consequently, we obtain the drilled manifolds in Table 2.

$X(k,1)$

k	Systole-drilled manifold
-7	$m081$
-6	$m011$
$-5,\dots,-2$	exceptional slopes
-1	$m004$
0	$m038$
otherwise	$m137$

$Y(k,1)$

k	Systole-drilled manifold
-9	$v3519$
-8	$v3200$
-7	$v2919$
-6	$v2911$
-5	$v3184$
-4	$v3505$
-3	not unique
otherwise	$t12072$

TABLE 2. Manifolds obtained by drilling the unique systolic geodesic in $X(k,1)$ and $Y(k,1)$.

We show that $Y(-3,1)$ contains exactly two systolic geodesics and we drill each of them. We thank Nathan Dunfield for this code.

```
import snappy
# Show that t12072(3,1) has exactly two systolic geodesics
# and that a self-isometry interchanges them
M = snappy.Manifold("t12072(3,1)")
S = M.symmetric_triangulation()
print("Verified beginning of the length spectrum of t12072(3,1):")
LS = S.length_spectrum_alt(count=4, verified=True, bits_prec=1000)
for g in LS:
    print(g)
print()
```

```
print("Drilling each systolic geodesic:")
for g in LS[:2]:
    T = S.copy()
    U = T.drill_word(g.word, verified=True, bits_prec=1000)
    print(f"{g.word:<20} {U.identify()}")
# Drill both systolic geodesics, yielding a 2-cusped hyperbolic 3-manifold.
S.dehn_fill([(0,0),(0,0)])
print()
print("Self-isometries of the drilled manifold:")
for iso in S.is_isometric_to(S, return_isometries=True):
    print(iso)
```

```
Verified beginning of the length spectrum of t12072(3,1):
Length                                      Core curve  Word
0.90188599874433... - 0.68042658939886...*I Cusp 1      fGGGGGG
0.90188599874433... - 0.68042658939886...*I Cusp 0      bcgdbCF
1.43277618896090... - 2.84374651337138...*I -           dE
1.61363312906004... - 2.06618680610597...*I -           aBEGCgcADGCgCgFbe
1.61363312906004... - 2.06618680610597...*I -           aBFggfcADe

Drilling each systolic geodesic:
fGGGGGG              [t12072(0,0)]
bcgdbCF              [t12072(0,0)]

Self-isometries of the drilled manifold:
0 -> 1   1 -> 0
[-1 -1]  [-1  1]
[ 0 -1]  [ 0 -1]
Extends to link
0 -> 0  1 -> 1
[1 0]   [1 0]
[0 1]   [0 1]
Extends to link
0 -> 0   1 -> 1
[-1  0]  [-1  0]
[ 0 -1]  [ 0 -1]
Extends to link
0 -> 1  1 -> 0
[1 1]   [1 -1]
[0 1]   [0  1]
Extends to link
0 -> 0   1 -> 1
[-1  0]  [-1  0]
```

```
[ 0 -1]  [ 0 -1]
Extends to link
0 -> 1  1 -> 0
[1 1]   [1 -1]
[0 1]   [0  1]
Extends to link
0 -> 1   1 -> 0
[-1 -1]  [-1  1]
[ 0 -1]  [ 0 -1]
Extends to link
0 -> 0  1 -> 1
[1 0]   [1 0]
[0 1]   [0 1]
Extends to link
```

The verified length spectrum of $t12072(3,1)$ begins with two geodesics, `Cusp 1` and `Cusp 0`, with apparently equal lengths. Drilling each of them yields $t12072$. Drilling both of them yields a two-cusped manifold with a self-isometry—the first isometry listed—interchanging the cusps and preserving the meridians up to orientation. Consequently, $t12072(3,1)$ has a self-isometry interchanging the geodesics `Cusp 1` and `Cusp 0`. Therefore, `Cusp 1` and `Cusp 0` are the two systolic geodesics of $t12072(3,1)$. It follows that $Y(-3,1)$ contains exactly two systolic geodesics and drilling each of them yields $t12072$.

We verify that the 12 drilled manifolds are pairwise nonhomeomorphic. By Mostow–Prasad rigidity, it suffices to show they are pairwise nonisometric.

```python
import snappy
# Verify the 12 systole drilled manifolds are distinct
names = ["m137", "t12072", "m081", "m011", "m004", "m038",
    "v3519", "v3200", "v2919", "v2911", "v3184", "v3505"]
manifolds = [snappy.Manifold(name) for name in names]
print("Checking pairwise isometry...")
duplicate = False
for i in range(len(manifolds)):
    for j in range(i + 1, len(manifolds)):
        if manifolds[i].is_isometric_to(manifolds[j]):
            duplicate = True
            print(f"{names[i]} is isometric to {names[j]}")
if not duplicate:
    print("All 12 manifolds are pairwise nonisometric.")
```

```
Checking pairwise isometry...
All 12 manifolds are pairwise nonisometric.
```

6. Volumes

We perform volume computations on the hyperbolic filled manifolds $X(k,1)$ and $Y(k,1)$ which suggest our conjectures on volume monotonicity and symmetry.

```
import snappy
# Volumes of Dehn fillings X(k,1) of X
X = snappy.Manifold('DT: [(14,12,-16,-10,18,-6),(-4,2,8)], [1,1,0,0,0,1,1,0,1]')
X.dehn_fill((0,1),1)
X = X.filled_triangulation()
Xid = X.identify()
print("Mazur knot exterior X")
print("Volume =", X.volume())
print("Isometric to census manifold:", Xid, "\n")
M = snappy.Manifold("m137")
print("Volumes of some integral Dehn fillings:\n")
print(f"{'k':^5}  {'vol(X(k,1r))':>15}  {'vol(m137(k+3r,1r))':>15}  {'equal?':>7}")
print("-" * 50)
k_values = list(range(-1000, -994)) + list(range(-20, 21)) + list(range(994, 1001))
for k in k_values:
    if k in {-5, -4, -3, -2}:
        print(f"{k:5d}  {'nonhyperbolic':>15}")
        continue
    DFX = X.copy()
    DFX.dehn_fill((k,1),0)
    vDFX = DFX.volume()
    DFM = M.copy()
    DFM.dehn_fill((k+3,1),0)
    vDFM = DFM.volume()
    equal = abs(vDFX - vDFM) < 1e-10
    print(f"{k:5d}  {vDFX:15.10f}  {vDFM:15.10f} {str(equal):>7}")
print()
print("Verify vol(X(k,1)) strictly increases as k increases from -1 to 1000:")
flag = True
X1 = X.copy()
X1.dehn_fill((-1,1),0)
v1 = X1.volume()
for k in range(0,1001):
    X2 = X.copy()
    X2.dehn_fill((k,1),0)
    v2 = X2.volume()
    #print(f"{k} {v1} {v2}")
    if v1 < v2:
        v1 = v2
```

```
        continue
    else:
        flag = False
        print(f"Volume stopped increasing at k = {k}")
        break
print(f"{flag}")
print()
print("Verify vol(X(k,1)) strictly increases as k decreases from -6 to -1000:")
flag = True
X1 = X.copy()
X1.dehn_fill((-6,1),0)
v1 = X1.volume()
for k in range(-7,-1001,-1):
    X2 = X.copy()
    X2.dehn_fill((k,1),0)
    v2 = X2.volume()
    #print(f"{k} {v1} {v2}")
    if v1 < v2:
        v1 = v2
        continue
    else:
        flag = False
        print(f"Volume stopped increasing at k = {k}")
        break
print(f"{flag}")
```

```
Mazur knot exterior X
Volume = 3.66386237670888
Isometric to census manifold: [m137(0,0)]

Volumes of some integral Dehn fillings:

  k        vol(X(k,1))  vol(m137(k+3,1))   equal?
-------------------------------------------------
-1000     3.6638400252     3.6638400252    True
 -999     3.6638399803     3.6638399803    True
 -998     3.6638399352     3.6638399352    True
 -997     3.6638398900     3.6638398900    True
 -996     3.6638398447     3.6638398447    True
 -995     3.6638397992     3.6638397992    True
  -20     3.5861901213     3.5861901213    True
  -19     3.5762257574     3.5762257574    True
  -18     3.5642301473     3.5642301473    True
```

-17	3.5496168070	3.5496168070	True
-16	3.5315737988	3.5315737988	True
-15	3.5089527264	3.5089527264	True
-14	3.4800896428	3.4800896428	True
-13	3.4425070875	3.4425070875	True
-12	3.3924017384	3.3924017384	True
-11	3.3237331543	3.3237331543	True
-10	3.2265517707	3.2265517707	True
-9	3.0838610497	3.0838610497	True
-8	2.8656302333	2.8656302333	True
-7	2.5162213804	2.5162213804	True
-6	1.9122102501	1.9122102501	True
-5	nonhyperbolic		
-4	nonhyperbolic		
-3	nonhyperbolic		
-2	nonhyperbolic		
-1	1.3985088842	1.3985088842	True
0	2.2597671326	2.2597671326	True
1	2.7124588084	2.7124588084	True
2	2.9868370451	2.9868370451	True
3	3.1623672864	3.1623672864	True
4	3.2795522103	3.2795522103	True
5	3.3608908541	3.3608908541	True
6	3.4193302144	3.4193302144	True
7	3.4625868297	3.4625868297	True
8	3.4954325756	3.4954325756	True
9	3.5209246919	3.5209246919	True
10	3.5410862256	3.5410862256	True
11	3.5572951021	3.5572951021	True
12	3.5705142075	3.5705142075	True
13	3.5814318669	3.5814318669	True
14	3.5905501777	3.5905501777	True
15	3.5982420594	3.5982420594	True
16	3.6047890027	3.6047890027	True
17	3.6104066018	3.6104066018	True
18	3.6152621701	3.6152621701	True
19	3.6194871148	3.6194871148	True
20	3.6231857749	3.6231857749	True
994	3.6638400476	3.6638400476	True
995	3.6638400923	3.6638400923	True
996	3.6638401369	3.6638401369	True
997	3.6638401813	3.6638401813	True
998	3.6638402256	3.6638402256	True

```
  999     3.6638402698     3.6638402698    True
 1000     3.6638403139     3.6638403139    True

Verify vol(X(k,1)) strictly increases as k increases from -1 to 1000:
True

Verify vol(X(k,1)) strictly increases as k decreases from -6 to -1000:
True
```

Consequently, we conjecture that:

- $\mathrm{vol}\,(X(k,1))$ strictly increases as k increases beginning at -1
- $\mathrm{vol}\,(X(k,1))$ strictly increases as k decreases beginning at -6
- $X(-1,1)$ has the minimum volume among all hyperbolic $X(k,1)$

```python
import snappy
# Volumes of Dehn fillings Y(k,1) of Y
Y = snappy.Manifold('DT: [(10,-20,-16,24,-14,22,18,-8,-4),(6,12,-2)],'
    '[1,1,0,1,0,0,1,1,1,0,1,0]')
Y.dehn_fill((0,1),1)
Y = Y.filled_triangulation()
Yid = Y.identify()
print("Jester knot exterior Y")
print("Volume =", Y.volume())
print("Isometric to census manifold:", Yid, "\n")
T = snappy.Manifold("t12072")
print("Volumes of some integral Dehn fillings:\n")
print(f"{'k':^5}  {'vol(Y(k,1r))':>15}  {'vol(t12072(k+6r,1r))':>18}  {'equal?':>7}")
print("-" * 53)
k_values = list(range(-1000, -994)) + list(range(-20, 21)) + list(range(994, 1001))
for k in k_values:
    DFY = Y.copy()
    DFY.dehn_fill((k,1),0)
    vDFY = DFY.volume()
    DFT = T.copy()
    DFT.dehn_fill((k+6,1),0)
    vDFT = DFT.volume()
    equal = abs(vDFY - vDFT) < 1e-10
    print(f"{k:5d}  {vDFY:15.10f}  {vDFT:18.10f} {str(equal):>7}")
print()
print("Verify vol(Y(k,1)) strictly increases as k increases from -6 to 1000:")
flag = True
Y1 = Y.copy()
Y1.dehn_fill((-6,1),0)
v1 = Y1.volume()
```

```
for k in range(-5,1001):
    Y2 = Y.copy()
    Y2.dehn_fill((k,1),0)
    v2 = Y2.volume()
    #print(f"{k} {v1} {v2}")
    if v1 < v2:
        v1 = v2
        continue
    else:
        flag = False
        print(f"Volume stopped increasing at k = {k}")
        break
print(f"{flag}")
print()
print("Verify vol(Y(k,1)) strictly increases as k decreases from -7 to -1000:")
flag = True
Y1 = Y.copy()
Y1.dehn_fill((-7,1),0)
v1 = Y1.volume()
for k in range(-8,-1001,-1):
    Y2 = Y.copy()
    Y2.dehn_fill((k,1),0)
    v2 = Y2.volume()
    #print(f"{k} {v1} {v2}")
    if v1 < v2:
        v1 = v2
        continue
    else:
        flag = False
        print(f"Volume stopped increasing at k = {k}")
        break
print(f"{flag}")
```

```
Jester knot exterior Y
Volume = 7.32772475341775
Isometric to census manifold: [t12072(0,0)]

Volumes of some integral Dehn fillings:

  k        vol(Y(k,1))  vol(t12072(k+6,1))   equal?
-------------------------------------------------------
-1000     7.3276797581        7.3276797581    True
 -999     7.3276796674        7.3276796674    True
```

-998	7.3276795764	7.3276795764	True
-997	7.3276794851	7.3276794851	True
-996	7.3276793936	7.3276793936	True
-995	7.3276793018	7.3276793018	True
-20	7.1085194314	7.1085194314	True
-19	7.0762146287	7.0762146287	True
-18	7.0366039004	7.0366039004	True
-17	6.9874700845	6.9874700845	True
-16	6.9257790105	6.9257790105	True
-15	6.8473580349	6.8473580349	True
-14	6.7464811852	6.7464811852	True
-13	6.6154126726	6.6154126726	True
-12	6.4441579271	6.4441579271	True
-11	6.2213963155	6.2213963155	True
-10	5.9406422204	5.9406422204	True
-9	5.6250329931	5.6250329931	True
-8	5.3589516115	5.3589516115	True
-7	5.2183624790	5.2183624790	True
-6	5.2183624790	5.2183624790	True
-5	5.3589516115	5.3589516115	True
-4	5.6250329931	5.6250329931	True
-3	5.9406422204	5.9406422204	True
-2	6.2213963155	6.2213963155	True
-1	6.4441579271	6.4441579271	True
0	6.6154126726	6.6154126726	True
1	6.7464811852	6.7464811852	True
2	6.8473580349	6.8473580349	True
3	6.9257790105	6.9257790105	True
4	6.9874700845	6.9874700845	True
5	7.0366039004	7.0366039004	True
6	7.0762146287	7.0762146287	True
7	7.1085194314	7.1085194314	True
8	7.1351524823	7.1351524823	True
9	7.1573309124	7.1573309124	True
10	7.1759714422	7.1759714422	True
11	7.1917723785	7.1917723785	True
12	7.2052715722	7.2052715722	True
13	7.2168877176	7.2168877176	True
14	7.2269500639	7.2269500639	True
15	7.2357200093	7.2357200093	True
16	7.2434069574	7.2434069574	True
17	7.2501800825	7.2501800825	True
18	7.2561771445	7.2561771445	True

```
    19      7.2615111597        7.2615111597    True
    20      7.2662754930        7.2662754930    True
   994      7.3276803855        7.3276803855    True
   995      7.3276804740        7.3276804740    True
   996      7.3276805623        7.3276805623    True
   997      7.3276806504        7.3276806504    True
   998      7.3276807381        7.3276807381    True
   999      7.3276808256        7.3276808256    True
  1000      7.3276809129        7.3276809129    True

Verify vol(Y(k,1)) strictly increases as k increases from -6 to 1000:
True

Verify vol(Y(k,1)) strictly increases as k decreases from -7 to -1000:
True
```

Consequently, we conjecture that:

- $\mathrm{vol}\,(Y(k,1))$ strictly increases as k increases beginning at -6
- $\mathrm{vol}\,(Y(k,1))$ strictly increases as k decreases beginning at -7
- $Y(-6,1)$ and $Y(-7,1)$ have the minimum volume among all $Y(k,1)$

```python
import snappy
# Jester volume symmetry vol(Y(k,1))=vol(Y(-k-13,1))
# vol(t12072(s,1))=vol(t12072(-s-1,1))
Y = snappy.Manifold('DT: [(10,-20,-16,24,-14,22,18,-8,-4),(6,12,-2)],'
    '[1,1,0,1,0,0,1,1,1,0,1,0]')
Y.dehn_fill((0,1),1)
Y = Y.filled_triangulation()
Yid = Y.identify()
print("Jester knot exterior Y")
print("Volume =", Y.volume())
print("Isometric to census manifold:", Yid, "\n")
#T = snappy.Manifold("t12072")
print("Volumes of some integral Dehn fillings:\n")
print(f"{'k':^5}  {'vol(Y(k,1r))':>15}  {'vol(Y(-k-13r,1r))':>18}  {'equal?':>7}")
print("-" * 52)
k_values = list(range(-1000, -994)) + list(range(-20, 21)) + list(range(994, 1001))
for k in k_values:
    Y1 = Y.copy()
    Y1.dehn_fill((k,1),0)
    v1 = Y1.volume()
    Y2 = Y.copy()
    Y2.dehn_fill((-k-13,1),0)
    v2 = Y2.volume()
```

```
    equal = abs(v1 - v2) < 1e-10
    print(f"{k:5d}  {v1:15.10f}  {v2:18.10f} {str(equal):>7}")
print()
print("Verify vol(Y(k,1))=vol(Y(-k-13,1)) for -1000<=k<=1000:")
flag = True
for k in range(-1000,1001):
    Y1 = Y.copy()
    Y1.dehn_fill((k,1),0)
    v1 = Y1.volume()
    Y2 = Y.copy()
    Y2.dehn_fill((-k-13,1),0)
    v2 = Y2.volume()
    equal = abs(v1 - v2) < 1e-10
    #print(f"{k} {v1} {v2}")
    if equal:
        continue
    else:
        flag = False
        print(f"Volumes unequal at k = {k}")
        break
print(f"{flag}")
```

```
Jester knot exterior Y
Volume = 7.32772475341775
Isometric to census manifold: [t12072(0,0)]

Volumes of some integral Dehn fillings:

  k        vol(Y(k,1))     vol(Y(-k-13,1))   equal?
-------------------------------------------------------
-1000     7.3276797581        7.3276797581    True
 -999     7.3276796674        7.3276796674    True
 -998     7.3276795764        7.3276795764    True
 -997     7.3276794851        7.3276794851    True
 -996     7.3276793936        7.3276793936    True
 -995     7.3276793018        7.3276793018    True
  -20     7.1085194314        7.1085194314    True
  -19     7.0762146287        7.0762146287    True
  -18     7.0366039004        7.0366039004    True
  -17     6.9874700845        6.9874700845    True
  -16     6.9257790105        6.9257790105    True
  -15     6.8473580349        6.8473580349    True
  -14     6.7464811852        6.7464811852    True
```

```
   -13     6.6154126726        6.6154126726    True
   -12     6.4441579271        6.4441579271    True
   -11     6.2213963155        6.2213963155    True
   -10     5.9406422204        5.9406422204    True
    -9     5.6250329931        5.6250329931    True
    -8     5.3589516115        5.3589516115    True
    -7     5.2183624790        5.2183624790    True
    -6     5.2183624790        5.2183624790    True
    -5     5.3589516115        5.3589516115    True
    -4     5.6250329931        5.6250329931    True
    -3     5.9406422204        5.9406422204    True
    -2     6.2213963155        6.2213963155    True
    -1     6.4441579271        6.4441579271    True
     0     6.6154126726        6.6154126726    True
     1     6.7464811852        6.7464811852    True
     2     6.8473580349        6.8473580349    True
     3     6.9257790105        6.9257790105    True
     4     6.9874700845        6.9874700845    True
     5     7.0366039004        7.0366039004    True
     6     7.0762146287        7.0762146287    True
     7     7.1085194314        7.1085194314    True
     8     7.1351524823        7.1351524823    True
     9     7.1573309124        7.1573309124    True
    10     7.1759714422        7.1759714422    True
    11     7.1917723785        7.1917723785    True
    12     7.2052715722        7.2052715722    True
    13     7.2168877176        7.2168877176    True
    14     7.2269500639        7.2269500639    True
    15     7.2357200093        7.2357200093    True
    16     7.2434069574        7.2434069574    True
    17     7.2501800825        7.2501800825    True
    18     7.2561771445        7.2561771445    True
    19     7.2615111597        7.2615111597    True
    20     7.2662754930        7.2662754930    True
   994     7.3276803855        7.3276803855    True
   995     7.3276804740        7.3276804740    True
   996     7.3276805623        7.3276805623    True
   997     7.3276806504        7.3276806504    True
   998     7.3276807381        7.3276807381    True
   999     7.3276808256        7.3276808256    True
  1000     7.3276809129        7.3276809129    True

Verify vol(Y(k,1))=vol(Y(-k-13,1)) for -1000<=k<=1000:
```

```
65 True
```

Consequently, we conjecture that:

- $\mathrm{vol}\,(Y(k,1)) = \mathrm{vol}\,(Y(-k-13,1))$ for all $k \in \mathbb{Z}$

7. Whitehead Link Exterior

The Whitehead link exterior W arises both as the exterior of a link in S^3 and a link in $S^1 \times S^2$. That topological observation is crucial to Section 3 of [CD26]. In Section 6 of [CD26], the cusp shapes of $W \subset S^1 \times S^2$ were computed geometrically. The SnapPy computations below provide independent verification. We first import the relevant link diagrams into SnapPy using the PLink Editor.

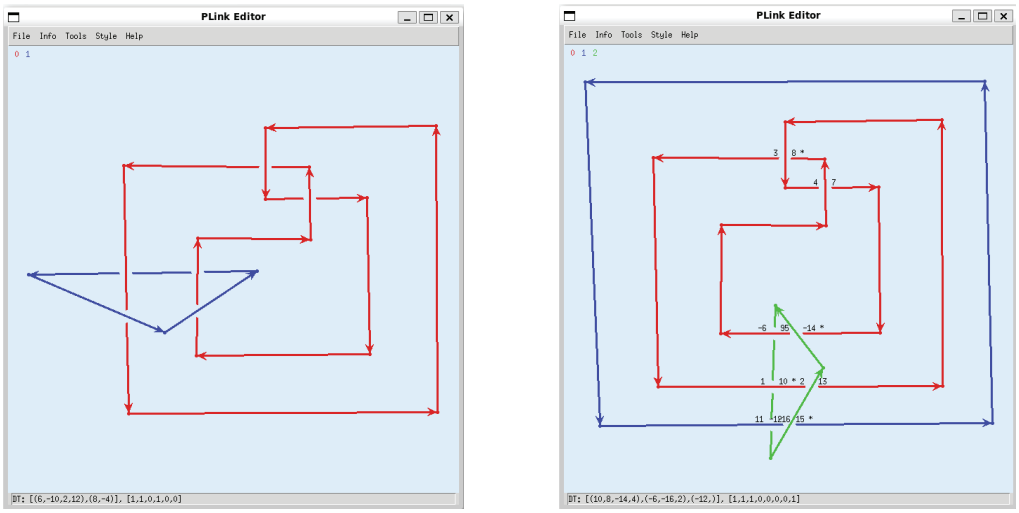

Figure 3. PLink Editor link diagrams for the Whitehead link exterior in S^3 (left) and in $S^1 \times S^2$ (right), together with their DT codes at the bottom.

The Dowker–Thistlethwaite (DT) codes for the two links in Figure 3 are

- `DT: [(6,-10,2,12),(8,-4)], [1,1,0,1,0,0]`
- `DT: [(10,8,-14,4),(-6,-16,2),(-12,)], [1,1,1,0,0,0,0,1]`

The former is a standard Whitehead link in S^3, and we refer to its exterior as $WS3$. The latter is a three-component link in S^3. Dehn-filling the green triangle with $(0,1)$ yields $S^1 \times S^2$, and W denotes the exterior of the other two components in $S^1 \times S^2$. This is the one place in our work in which care must be taken regarding the order of link components. The components of our PLink Editor diagram in Figure 3 are ordered 0 red, 1 blue, and 2 green. However, the DT code swaps the order of the blue and green components due to DT conventions. That is visible by inspection in the PLink Editor by clicking on `Info` and `DT labels`. So, SnapPy reads the DT code with red as cusp 0, green as cusp 1, and blue as cusp 2. Therefore, in SnapPy we must perform $(0,1)$ Dehn-surgery on cusp 1 to obtain W.

We compute basic information about $WS3$.

```
1 import snappy
2 # Whitehead link exterior WS3 in S^3
```

```
WS3 = snappy.Manifold('DT: [(6,-10,2,12),(8,-4)], [1,1,0,1,0,0]')
WS3id = WS3.identify()
IsomWS3 = WS3.symmetry_group()
print("Whitehead link exterior WS3 in S^3")
print("Volume =", WS3.volume())
print("Isometric to census manifold:", WS3id)
print("Cusp shape =", WS3.cusp_info('shape'))
print("")
print("Is WS3 achiral?", IsomWS3.is_amphicheiral())
print("Order of isometry group =", IsomWS3.order())
print("Isometry group =", IsomWS3)
print("Self-isometries are:", IsomWS3.isometries())
```

```
Whitehead link exterior WS3 in S^3
Volume = 3.66386237670888
Isometric to census manifold: [m129(0,0)(0,0), 5^2_1(0,0)(0,0), L5a1(0,0)(0,0),
    ooct01_00001(0,0)(0,0)]
Cusp shape = [2.00000000000000 + 2.00000000000000*I,
    2.00000000000000 + 2.00000000000000*I]

Is WS3 achiral? False
Order of isometry group = 8
Isometry group = D4
Self-isometries are: [0 -> 0  1 -> 1
[1 0]   [1 0]
[0 1]   [0 1]
Extends to link, 0 -> 1   1 -> 0
[-1  0]  [1 0]
[ 0 -1]  [0 1]
Extends to link, 0 -> 0   1 -> 1
[-1  0]  [-1  0]
[ 0 -1]  [ 0 -1]
Extends to link, 0 -> 1  1 -> 0
[1 0]   [-1  0]
[0 1]   [ 0 -1]
Extends to link, 0 -> 1   1 -> 0
[-1  0]  [-1  0]
[ 0 -1]  [ 0 -1]
Extends to link, 0 -> 0  1 -> 1
[1 0]   [-1  0]
[0 1]   [ 0 -1]
Extends to link, 0 -> 1  1 -> 0
[1 0]   [1 0]
```

```
[0 1]   [0 1]
Extends to link, 0 -> 0   1 -> 1
[-1  0]  [1 0]
[ 0 -1]  [0 1]
Extends to link]
```

We compute basic information about W.

```
import snappy
# Whitehead link exterior W in S^1xS^2
# DT conventions swap the order of the blue and green components
# Correspondingly, we Dehn-fill cusp 1 with (0,1) to get S^1xS^2
W = snappy.Manifold('DT: [(10,8,-14,4),(-6,-16,2),(-12,)], [1,1,1,0,0,0,0,1]')
W.dehn_fill((0,1),1)
W = W.filled_triangulation()
Wid = W.identify()
IsomW = W.symmetry_group()
print("Whitehead link exterior W in S^1xS^2")
print("Volume =", W.volume())
print("Isometric to census manifold:", Wid)
print("Cusp shape =", W.cusp_info('shape'))
print("")
print("Is W achiral?", IsomW.is_amphicheiral())
print("Order of isometry group =", IsomW.order())
print("Isometry group =", IsomW)
print("Self-isometries are:", IsomW.isometries())
```

```
Whitehead link exterior W in S^1xS^2
Volume = 3.66386237670888
Isometric to census manifold: [m129(0,0)(0,0), 5^2_1(0,0)(0,0), L5a1(0,0)(0,0),
    ooct01_00001(0,0)(0,0)]
Cusp shape = [2.00000000000000 + 2.00000000000000*I,
    -0.250000000000000 + 0.250000000000000*I]

Is W achiral? False
Order of isometry group = 8
Isometry group = D4
Self-isometries are: [0 -> 0  1 -> 1
[1 0]   [1 0]
[0 1]   [0 1]
Extends to link, 0 -> 1  1 -> 0
[0 -1]  [0 -1]
[1  0]  [1  0]
```

```
Does not extend to link, 0 -> 0   1 -> 1
[-1  0]  [-1  0]
[ 0 -1]  [ 0 -1]
Extends to link, 0 -> 1  1 -> 0
[ 0 1]  [ 0 1]
[-1 0]  [-1 0]
Does not extend to link, 0 -> 1  1 -> 0
[0 -1]  [ 0 1]
[1  0]  [-1 0]
Does not extend to link, 0 -> 0  1 -> 1
[1 0]   [-1  0]
[0 1]   [ 0 -1]
Extends to link, 0 -> 1  1 -> 0
[ 0 1]  [0 -1]
[-1 0]  [1  0]
Does not extend to link, 0 -> 0   1 -> 1
[-1  0]  [1 0]
[ 0 -1]  [0 1]
Extends to link]
```

Those cusp shapes agree with our geometric arguments in Section 6 of [CD26].

References

[CD26] Jack S. Calcut and Yangyang Du. Mazur's knot and the octahedron. https://arxiv.org/abs/2606.17335, 2026.

[Dun20] Nathan M. Dunfield. A census of exceptional Dehn fillings. In *Characters in low-dimensional topology*, volume 760 of *Contemp. Math.*, pages 143–155. Amer. Math. Soc., [Providence], RI, [2020] ©2020.

[FPS25] David Futer, Jessica S. Purcell, and Saul Schleimer. Excluding cosmetic surgeries on hyperbolic 3-manifolds. *Journal of Computational Geometry*, 16(1):694–736, 2025.

Department of Mathematics, Oberlin College
Email address: jcalcut@oberlin.edu

Department of Mathematics, University of Michigan
Email address: yyadu@umich.edu